

Definir estándares de codificación de acuerdo con la plataforma de desarrollo elegida
GA7-220501096-AA1-EV02

Nombre
Hector Wanderley Palencia Puentes

Instructor
Oscar Fernando Carvajal Castaño

Centro de Teleinformática y Producción Industrial
Análisis y Desarrollo de Software
Ficha: 3186640
2026

El presente informe técnico tiene como propósito documentar las herramientas y tecnologías de versionamiento seleccionadas para el desarrollo del sistema SIAPE, una aplicación web orientada a la gestión integral de inventarios, ventas, proveedores y facturación para pequeñas y medianas empresas.

El control de versiones es un componente fundamental en cualquier proyecto de desarrollo de software, ya que permite registrar el historial de cambios del código fuente, facilitar la colaboración entre miembros del equipo, revertir modificaciones no deseadas y mantener versiones estables del sistema en todo momento.

Dado que SIAPE es un sistema compuesto por nueve módulos funcionales (Pedidos, Inventario, Proveedores, Clientes, Bodegas, Trabajadores, Facturas, Pagos y Reportes) desarrollados sobre un stack de React, Node.js y MySQL, la elección de herramientas de versionamiento adecuadas resulta crítica para garantizar la trazabilidad, la calidad y la mantenibilidad del software a lo largo de su ciclo de vida.

Sumado a lo anterior también se define los estándares de codificación que se aplicarán durante el desarrollo del sistema SIAPE. Un estándar de codificación es un conjunto de reglas y convenciones acordadas que regulan la forma en que se escribe el código fuente, con el objetivo de garantizar su legibilidad, consistencia, mantenibilidad y calidad a lo largo del ciclo de vida del proyecto.

SIAPE estará construido sobre un stack compuesto por React en el frontend, Node.js en el backend y MySQL como sistema de gestión de base de datos, este documento establece estándares específicos para cada capa tecnológica, abarcando la nomenclatura de variables, funciones y componentes, la organización de archivos, el manejo de errores, el formato del código y las convenciones para la base de datos.

1. Objetivo

- 1.1** Definir y justificar las herramientas y tecnologías de versionamiento que se utilizarán durante el desarrollo del sistema SIAPE, tomando como base las características técnicas del proyecto, el stack tecnológico seleccionado y las buenas prácticas de la industria del desarrollo de software.
- 1.2** Definir el conjunto de reglas, convenciones y buenas prácticas de codificación que se aplicarán en el desarrollo del sistema SIAPE, cubriendo las tres capas del stack tecnológico (React, Node.js y MySQL), con el fin de garantizar un código fuente limpio, coherente y fácil de mantener.

2. Link al repositorio de github: <https://github.com/0wander1/SIAPE.git>

3. Selección Justificada de Herramientas

3.1 Git — Sistema de Control de Versiones

Git es el sistema de control de versiones distribuido más utilizado en la industria. Permite a los desarrolladores llevar un registro detallado de todos los cambios realizados sobre el código fuente, trabajar en paralelo mediante ramas (branches) y fusionar cambios de forma controlada.

Justificación para SIAPE:

- El proyecto está compuesto por múltiples módulos independientes (frontend en React, backend en Node.js, base de datos en MySQL), lo que hace necesario un sistema que permita manejar cambios en diferentes capas del stack de forma simultánea.
- Git permite crear ramas específicas por módulo o funcionalidad (por ejemplo: feature/modulo-pedidos, feature/modulo-facturas), aislando el desarrollo de cada uno sin afectar el código estable.
- Su naturaleza distribuida garantiza que el historial completo del proyecto esté disponible localmente, permitiendo trabajar sin conexión a internet cuando sea necesario.

3.2 GitHub — Plataforma de Alojamiento de Repositorios

GitHub es la plataforma en la nube más popular para alojar repositorios Git. Ofrece funcionalidades adicionales como revisión de código (pull requests), seguimiento de incidencias (issues), integración continua y documentación integrada mediante wikis y archivos README.

Justificación para SIAPE:

- Permite centralizar el repositorio del proyecto en la nube, garantizando un respaldo permanente del código fuente.
- Los pull requests permiten revisar y aprobar cambios antes de integrarlos a la rama principal, reduciendo la introducción de errores en el código base.
- La integración con herramientas de CI/CD como GitHub Actions facilita la automatización de pruebas y despliegues en etapas futuras del proyecto.
- El sistema de issues permite registrar y hacer seguimiento a los errores, tareas pendientes y mejoras del sistema de forma organizada.

3.3 Estrategia de Ramas (Git Flow)

Para el desarrollo de SIAPE se adoptará una estrategia de ramas basada en Git Flow, adaptada a las necesidades de un proyecto de un solo desarrollador pero estructurada para escalar. La estructura de ramas será la siguiente:

Rama	Propósito
main	Versión estable y lista para producción. Solo recibe merges desde develop una vez validada.
develop	Rama de integración principal. Contiene el código en desarrollo activo
feature/*	Ramas para desarrollo de funcionalidades específicas. Ej: feature/modulo-inventario.
fix/*	Ramas para corrección de errores. Ej: fix/calculo-total-factura.
release/*	Ramas de preparación para un nuevo release antes de pasar a main

3.4 Convención de Commits (Conventional Commits)

Se adoptará el estándar Conventional Commits para los mensajes de commit, lo que facilita la lectura del historial y la generación automática de changelogs. El formato es el siguiente:

```
<tipo>(<alcance>): <descripción breve>
```

Los tipos de commit que se utilizarán son:

- **feat:** para la adición de nuevas funcionalidades. Ej: feat(pedidos): agregar modal de nuevo pedido.
- **fix:** para corrección de errores. Ej: fix(facturas): corregir cálculo de impuesto.
- **refactor:** para reestructuración de código sin cambio de funcionalidad.
- **docs:** para cambios en documentación.
- **style:** para cambios de formato o estilos visuales sin afectar la lógica.
- **chore:** para tareas de mantenimiento como actualización de dependencias.

3.5 Herramientas Complementarias

.gitignore: Archivo de configuración que excluye del repositorio archivos innecesarios o sensibles como node_modules/, archivos .env con variables de entorno (contraseñas de base de datos, claves API) y archivos de configuración local del editor.

README.md: Documento en la raíz del repositorio que describirá el proyecto, los requisitos del sistema, las instrucciones de instalación y ejecución del stack React + Node.js + MySQL, y la estructura de carpetas del proyecto.

Variables de entorno (.env): Las credenciales de conexión a la base de datos MySQL y otras configuraciones sensibles se gestionarán mediante archivos .env, asegurando que nunca sean versionados en el repositorio.

4. Estándares de Codificación

4.1 Estándares Generales (JavaScript / React / Node.js)

3.1.1 Nomenclatura de variables y constantes

Se usará camelCase para todas las variables y constantes de valor dinámico, y UPPER_SNAKE_CASE para constantes de valor fijo:

```
// Correcto const totalFactura = 50000; const nombreProveedor = 'TechSupply';  
const MAX_INTENTOS_LOGIN = 3; // Incorrecto const TotalFactura = 50000; const  
nombre_proveedor = 'TechSupply';
```

```
const btnNuevoPedido = document.getElementById('btnNuevoPedido');
```

3.1.2 Nomenclatura de funciones

Las funciones se nombrarán en camelCase comenzando con un verbo que describa su acción:

```
// Correcto function calcularTotalFactura() { ... } function  
obtenerPedidosPorCliente(idCliente) { ... } function  
validarCredencialesAdmin(usuario, clave) { ... } // Incorrecto function  
factura() { ... } function datos() { ... }
```

3.1.3 Uso de const y let

- Se usará const por defecto para toda declaración de variable.
- Se usará let únicamente cuando el valor de la variable deba reasignarse posteriormente.
- Queda prohibido el uso de var en todo el proyecto.

```
const menuItems = document.querySelectorAll('.sidebar li');
```

3.1.4 Formato y estilo de código

- Indentación de 2 espacios en todo el proyecto (no tabs).
- Punto y coma al final de cada sentencia.
- Comillas simples para strings en JavaScript/Node.js.
- Longitud máxima de línea: 200 caracteres.

```
menuItems.forEach(i => i.classList.remove('active'));
```

3.2 Estándares para React (Frontend)

3.2.1 Nomenclatura de componentes

Los componentes de React se nombrarán en PascalCase. El nombre debe describir claramente qué representa el componente:

```
// Correcto function NuevoPedidoModal() { ... } function TablaInventario() { ...  
} function TarjetaTrabajador() { ... } // Incorrecto function modal() { ... }  
function tabla_inventario() { ... }
```

3.2.2 Nomenclatura de archivos de componentes

- Un archivo por componente.
- El nombre del archivo debe coincidir exactamente con el nombre del componente.
- Extensión .jsx para componentes de React.

NuevoPedidoModal.jsx TablaInventario.jsx TarjetaTrabajador.jsx

3.2.3 Estructura de carpetas del frontend

```
src/  
  components/  
  pages/ pedidos/ inventario/ proveedores/ ...  
  hooks/  
  services/  
  utils/  
  assets/
```

3.2.4 Props y manejo de estado

- Las props se nombrarán en camelCase y deben ser descriptivas del dato que representan.
- Se usará useState para estado local del componente y useEffect para efectos secundarios.
- Se evitará el prop drilling de más de dos niveles; en ese caso se usará Context API.

3.3 Estándares para Node.js (Backend)

3.3.1 Estructura de carpetas del backend

```
src/  
routes/ pedidos.routes.js inventario.routes.js ...  
controllers/ pedidos.controller.js  
services/ pedidos.service.js  
models/  
middlewares/  
config/  
utils/
```

3.3.2 Nomenclatura de archivos del backend

- Se usará kebab-case para nombres de archivos: pedidos.controller.js, auth.middleware.js.
- Cada archivo debe cumplir una única responsabilidad (principio de responsabilidad única).

3.3.3 Rutas de la API

Las rutas seguirán convenciones REST. Se usarán sustantivos en plural y minúsculas:

```
GET /api/pedidos # Obtener todos los pedidos  
GET /api/pedidos/:id # Obtener un pedido por ID  
POST /api/pedidos # Crear un nuevo pedido  
PUT /api/pedidos/:id # Actualizar un pedido  
DELETE /api/pedidos/:id # Eliminar un pedido
```

3.3.4 Manejo de errores

- Todo bloque asíncrono debe estar envuelto en try/catch.
- Los errores deben retornar un objeto JSON con los campos message y statusCode.
- Se centralizará el manejo de errores en un middleware global de Express.

```
// Respuesta de error estándar res.status(400).json({ statusCode: 400, message:  
'El campo cliente es obligatorio' });
```

3.4 Estándares para MySQL (Base de Datos)

3.4.1 Nomenclatura de tablas

- Nombres en snake_case y en singular: pedido_externo, movimiento_inventario.
- Las tablas asociativas se nombran combinando las dos entidades: producto_has_proveedor.

3.4.2 Nomenclatura de columnas

- Todas las columnas en snake_case: fecha_emision, valor_total, cantidad_disponible.
- Las llaves primarias se nombran id_: id_pedido, id_factura, id_bodega.
- Las llaves foráneas incluyen el nombre de la tabla referenciada: cliente_id_usuario_cli, producto_id_producto.

3.4.3 Escritura de consultas SQL

- Las palabras clave de SQL se escriben en MAYÚSCULAS: SELECT, FROM, WHERE, JOIN, INSERT INTO.
- Los nombres de tablas y columnas en minúsculas.
- Cada cláusula principal de la consulta va en una línea separada para facilitar la lectura.

```
-- Correcto SELECT p.id_pedido, p.valor_total, c.nombre_usuario FROM
pedido_externo p JOIN cliente c ON p.cliente_id_usuario_cli = c.id_usuario_cli
WHERE p.estado = 'pendiente' ORDER BY p.fecha_entrega_estimada ASC;
```

3.4.4 Buenas prácticas de seguridad

- Se usarán consultas parametrizadas en todas las operaciones desde Node.js para prevenir inyección SQL.
- Nunca se construirán consultas SQL concatenando strings con datos del usuario.

```
// Correcto (consulta parametrizada) const [rows] = await db.query( 'SELECT *
FROM pedido_externo WHERE id_pedido = ?', [idPedido] ); // Incorrecto
(vulnerable a SQL Injection) const query = 'SELECT * FROM pedido_externo WHERE
id_pedido = ' + idPedido;
```